

Linux

La ligne de commande · fichiers · droits · Bash · scripts · système · réseau · SSH

ls -l chasse fixe, gras : à taper tel quel
fichier italique : valeur à remplacer
sudo en tête : droits d'administration requis
Ctrl+C touche ou combinaison de touches
❗ piège fréquent ou commande risquée

ANATOMIE D'UNE COMMANDE

ls **-l -h** **/var/log**

COMMANDE OPTIONS ARGUMENT

- Options courtes groupables : **-l -h = -lh** ; forme longue : **--human-readable**.
- La casse compte : **ls -r** (ordre inverse) ≠ **ls -R** (récursif).
- clôt les options : **rm -- -fichier**.
- Espace dans un nom :
cd "Mes documents" ou
cd Mes\ documents.
- Code de retour : **0** = succès, toute autre valeur = échec → **echo \$?**.

L'INVITE DU SHELL

alice **@pc** **~/docs** **\$**
UTILISATEUR MACHINE DOSSIER COURANT INVITE

\$: utilisateur ordinaire · # : administrateur (root).

OBTENIR DE L'AIDE

man <i>commande</i>	manuel complet ; [q] quitter, /mot chercher
<i>commande</i> --help	aide résumée
man -k <i>mot</i>	chercher une commande par mot-clé (= apropos)
whatis <i>commande</i>	description en une ligne
type <i>commande</i>	alias, commande interne ou programme ?
which <i>commande</i>	chemin du programme lancé
help <i>commande</i>	commandes internes de Bash (cd , export ...)

Sections du manuel : **1** commandes · **5** formats de fichiers · **8** administration. Ex. **man 5 crontab**.

CHEMINS

/etc/ssh	absolu : part de la racine /
docs/a.txt	relatif : part du dossier courant
~	dossier personnel (= \$HOME)
· · ·	dossier courant · dossier parent
-	avec cd : dossier précédent
.bashrc	fichier caché : nom commençant par un point

❗ La casse compte : **Photo.jpg** ≠ **photo.jpg**.
Séparateur : **/** (et non ****).

PRATIQUER SANS RISQUE

- Machine virtuelle** avec instantanés : on revient en arrière après une erreur.
- WSL** sous Windows : pratique pour s'exercer, mais n'isole pas du système hôte.
- Clé USB « live »** : Linux démarre sans installation.
- Provoquer les erreurs **volontairement**, au laboratoire, pour apprendre à les lire.
- ❗ Ne jamais expérimenter avec **sudo** sur une machine dont on dépend.

RACCOURCIS DU TERMINAL

Tab	compléter un nom ; deux fois : propositions
[↑ ↓]	parcourir l'historique
Ctrl+R	rechercher dans l'historique
Ctrl+C	interrompt la commande en cours
Ctrl+D	fin de saisie ; sur ligne vide : quitter le shell
Ctrl+Z	suspendre (repandre : fg ou bg)
Ctrl+L	effacer l'écran (= clear)
Ctrl+A	début · fin de ligne
Ctrl+E	
Ctrl+U	couper jusqu'au début · jusqu'à la fin
Ctrl+K	
Ctrl+W	couper le mot précédent · coller
Ctrl+Y	
Alt+.	dernier argument de la commande précédente
Ctrl+Maj+C	copier · coller (terminal graphique)
Ctrl+Maj+V	
❗ Ctrl+S	fige l'affichage ; Ctrl+Q le libère.

ÉDITEURS : LE MINIMUM VITAL

NANO — SIMPLE

Ctrl+O	enregistrer
Entrée	
Ctrl+X	quitter
Ctrl+W	chercher
Ctrl+K	couper la ligne · coller
Ctrl+U	
Alt+U	annuler
Dans les menus de nano : ^ = Ctrl · M- = Alt	

VIM — PRÉSENT PARTOUT

[i]	mode insertion : saisir du texte
Échap	retour au mode normal (commandes)
:w · :q	enregistrer · quitter
:wq	enregistrer et quitter
:q!	quitter sans enregistrer
/motif · n	chercher · occurrence suivante
u · Ctrl+R	annuler · rétablir
dd · yy · p	couper · copier la ligne · coller
gg · G	début · fin du fichier
:set nu	numéros de ligne
❗ Coincé dans vim : Échap puis :q! Entrée .	

CLAVIER AZERTY : CARACTÈRES DU SHELL

 AltGr+6	~ AltGr+2	# AltGr+3
{ AltGr+4	[AltGr+5	` AltGr+7
\ AltGr+8	@ AltGr+0	] AltGr+9
} AltGr+=		
Touche morte (~, `) : ajouter Espace .		

ARBORESCENCE DU SYSTÈME

/	racine unique de tout le système
/bin	commandes essentielles
/sbin	(souvent liens vers /usr/bin , /usr/sbin)
/boot	noyau et chargeur de démarrage
/dev	périphériques : disques, terminaux...
/etc	configuration du système et des services
/home	dossiers personnels des utilisateurs
/root	dossier personnel de l'administrateur
/lib	bibliothèques, modules du noyau
/media	supports amovibles · montages temporaires
/mnt	
/opt	logiciels tiers autonomes
/proc	fichiers virtuels : processus, noyau, matériel
/sys	
/run	données d'exécution, effacées à l'arrêt
/srv	données servies (web, FTP...)
/tmp	fichiers temporaires
/usr	programmes, bibliothèques, documentation
/var	données variables : journaux (/var/log), caches

« Tout est fichier » : disques, processus et réglages du noyau se lisent comme des fichiers.

SE DÉPLACER ET LISTER

pwd	afficher le dossier courant
cd dossier	aller dans un dossier
cd	revenir au dossier personnel
cd ..	remonter d'un niveau
cd -	revenir au dossier précédent
ls -l	format long : droits, taille, date
ls -a	inclure les fichiers cachés
ls -lh	tailles lisibles (K, M, G)
ls -lt · ls -ltr	par date : récent en tête · en dernier
ls -ls	par taille décroissante
ls -R	récursif (sous-dossiers)
ls -d */	les dossiers seulement
tree -L 2	arbre sur 2 niveaux (paquet tree)

DÉMARCHE DE DIAGNOSTIC

- Lire le message **en entier** : la première erreur est souvent la vraie.
- Code de retour : **echo \$?**.
- Plus de détails : **-v**, **--verbose**, **bash -x**.
- Journaux : **journalctl -u service -e, /var/log/**.
- Vérifier l'évidence : chemin, droits, espace, service, réseau.
- Ne changer **qu'une chose à la fois**, puis retester.

2 · Fichiers et dossiers

CRÉER, COPIER, DÉPLACER, SUPPRIMER

touch <i>f</i>	créer un fichier vide (ou actualiser sa date)
mkdir <i>d</i>	créer un dossier
mkdir -p <i>a/b/c</i>	créer toute la hiérarchie d'un coup
cp <i>source destination</i>	copier un fichier
cp -r <i>d1 d2</i>	copier un dossier et son contenu
cp -a <i>source destination</i>	copie fidèle : droits, dates, liens conservés
cp -i · mv -i	demandeur avant d'écraser
cp -n	ne jamais écraser
mv <i>ancien nouveau</i>	renommer
mv <i>f1 f2 d/</i>	déplacer plusieurs fichiers dans un dossier
rm <i>f</i>	supprimer un fichier
rm -i <i>f</i>	supprimer avec confirmation
rm -r <i>d</i>	supprimer un dossier et tout son contenu
rmdir <i>d</i>	supprimer un dossier vide

❗ Pas de corbeille : **rm** est **définitif** et **rm -rf** ne demande rien. Vérifier d'abord le motif avec **ls** *motif*.

❗ **cp** et **mv** écrasent sans prévenir un fichier de même nom (sauf **-i** ou **-n**).

LIENS

ln -s <i>cible lien</i>	lien symbolique : un raccourci (peut pointer dans le vide)
ln <i>cible lien</i>	lien physique : second nom des mêmes données
ls -l <i>lien</i>	affiche <i>lien</i> → <i>cible</i>
readlink -f <i>lien</i>	chemin réel final

Ordre : **cible d'abord**, comme **cp source destination**. Lien physique : même système de fichiers, jamais un dossier.

JOKERS DU SHELL

*	toute suite de caractères, même vide	*.txt
?	exactement un caractère	photo?.jpg
[abc]	un caractère parmi a, b, c	[Rr]apport*
[0-9]	un caractère de l'intervalle	log[0-9]
[!0-9]	un caractère hors de l'ensemble	[!0-9]*
{a,b}	variantes	f{,.bak} → f f.bak
{1..5}	séquence	mkdir s{1..5}

C'est le **shell** qui remplace les jokers, avant de lancer la commande ; * ignore les fichiers cachés. Voir d'avance : **echo *.txt**.

COMPARER ET VÉRIFIER

diff <i>a b</i>	différences ligne à ligne
diff -u <i>a b</i>	format unifié : + ajouté, - retiré
diff -r <i>d1 d2</i>	comparer deux dossiers
sha256sum <i>f</i>	empreinte SHA-256 d'un fichier
sha256sum -c <i>f.sha256</i>	vérifier des empreintes publiées

Un téléchargement est intact si son empreinte est identique à celle que publie la source officielle.

ENCODAGE ET NOMS DE FICHIERS

file <i>f</i>	encodage et fins de ligne (« CRLF » = Windows)
dos2unix <i>f</i> · unix2dos <i>f</i>	convertir les fins de ligne (paquet dos2unix)
iconv -f ISO-8859-1 -t UTF-8 <i>f</i> > <i>g</i>	convertir l'encodage en UTF-8
locale	langue et encodage de la session

Noms sûrs : lettres sans accent, chiffres, -, _, . ; ni espace ni caractère spécial.

LIRE UN FICHIER

cat <i>f</i> · cat -n <i>f</i>	afficher tout · avec numéros de ligne
less <i>f</i>	lire page par page (touches ci-dessous)
head <i>f</i> · head -n 20 <i>f</i>	10 (ou 20) premières lignes
tail <i>f</i> · tail -n 20 <i>f</i>	10 (ou 20) dernières lignes
tail -f <i>f</i>	suivre en direct (journal) ; [Ctrl+C] pour sortir
wc -l <i>f</i>	compter les lignes (-w mots, -c octets)
file <i>f</i>	nature réelle du contenu
stat <i>f</i>	taille, droits, dates, inode

TOUCHES DE LESS (ET DE MAN)

[Espace] · [b]	page suivante · précédente
/mot · [n] · [N]	chercher · suivant · précédent
[g] · [G]	début · fin
[F]	suivre les ajouts (comme tail -f)
[q]	quitter

ARCHIVES ET COMPRESSION

tar -czf <i>a.tar.gz d/</i>	créer une archive compressée (gzip)
tar -cJf <i>a.tar.xz d/</i>	idem en xz : plus compact, plus lent
tar -tzf <i>a.tar.gz</i>	lister le contenu sans extraire
tar -xzf <i>a.tar.gz</i>	extraire ici
tar -xf <i>a.tar.gz -C d/</i>	extraire dans <i>d</i> (compression détectée)
gzip <i>f</i> · gunzip <i>f.gz</i>	compresser · décompresser (remplace l'original)
zip -r <i>a.zip d/</i>	archive zip
unzip <i>a.zip</i> · unzip -l <i>a.zip</i>	extraire · lister

Mémo **tar** : **c** créer · **x** extraire · **t** lister · **z** gzip · **J** xz · **j** bzip2 · **v** détails · **f** fichier — **f** en dernier, suivi du nom de l'archive.

DISQUES ET ESPACE

df -h	espace libre par système de fichiers
df -i	inodes libres (nombre de fichiers possibles)
du -sh <i>d</i>	taille totale d'un dossier
du -h --max-depth=1	taille de chaque sous-dossier
du -sh * sort -h	éléments du dossier courant, triés
lsblk -f · findmnt	disques et partitions · points de montage
sudo mount <i>/dev/sdb1 /mnt</i>	monter une partition
sudo umount <i>/mnt</i>	démonter (fermer d'abord les fichiers ouverts)

/etc/fstab : montages au démarrage. ❗ Une erreur peut bloquer le démarrage : contrôler avec **findmnt --verify**, puis **sudo mount -a**.

❗ **mkfs**, **dd**, **fdisk** sur le mauvais disque effacent tout : vérifier son nom avec **lsblk**, deux fois.

À MANIER AVEC PRÉCAUTION

rm -rf <i>chemin</i>	suppression définitive, sans question : relire le chemin
rm -rf "\$d"/*	❗ si <i>d</i> est vide → /* ! Écrire "\${d:?}"/*
dd · mkfs · fdisk	écrasent un disque entier en cas d'erreur de nom
chmod -R · chown -R	sur / ou <i>/etc</i> : système cassé, sécurité perdue
> <i>f</i>	vide le fichier sans confirmation
curl <i>url</i> sh	exécute un script jamais lu : télécharger, lire, puis lancer

Règle d'or : comprendre chaque option **avant** **[Entrée]** ; simuler d'abord (**--dry-run**, **-i**, **echo**) ; ne jamais coller une commande que l'on ne comprend pas.

3 · Rechercher et traiter le texte

MÉMENTO — LINUX

FIND — TROUVER DES FICHIERS

Syntaxe : `find` où *critères* *action*

<code>find . -name "*.log"</code>	par nom (casse respectée)
<code>find . -iname "photo*"</code>	par nom, casse ignorée
<code>find d -type f</code>	fichiers seulement (d dossiers, 1 liens)
<code>find . -size +100M</code>	plus de 100 Mio
<code>find . -mtime -7</code>	modifiés il y a moins de 7 jours
<code>find . -mtime +30</code>	modifiés il y a plus de 30 jours
<code>find . -mmin -60</code>	modifiés dans la dernière heure
<code>find . -user nom</code>	appartenant à un utilisateur
<code>find . -empty</code>	fichiers et dossiers vides
<code>find . -maxdepth 1</code>	sans descendre dans les sous-dossiers
<code>find . -name "*.sh" -exec chmod +x {} +</code>	agir sur les résultats
<code>find . -name "*.tmp" -delete</code>	❗ supprimer les résultats : lancer d'abord sans <code>-delete</code>
<code>find / -name f 2>/dev/null</code>	masquer les « Permission denied »

Combiner : `-a` et (implicite) · `-o` ou `!` non · `\( ... \)` groupe.❗ Motif entre guillemets, sinon le shell le remplace avant `find`.

GREP — CHERCHER DANS LE TEXTE

<code>grep motif f</code>	lignes contenant le motif
<code>grep -i</code>	ignorer la casse
<code>grep -v</code>	inverser : lignes sans le motif
<code>grep -n</code>	afficher les numéros de ligne
<code>grep -c</code>	compter les lignes trouvées
<code>grep -w</code>	mot entier seulement
<code>grep -o</code>	n'afficher que la partie trouvée
<code>grep -r motif d/</code>	chercher dans tout un dossier
<code>grep -rl motif d/</code>	seulement les noms de fichiers
<code>grep -E "a b"</code>	expression régulière étendue (a ou b)
<code>grep -F "a.b"</code>	texte littéral : aucun caractère spécial
<code>grep -A 3 · -B 3 · -C 3</code>	3 lignes après · avant · autour
<code>grep -q</code>	silencieux : seul le code de retour compte (scripts)

RECETTES : COMPOSER AVEC LE TUBE |

<code>history grep ssh</code>	retrouver une ancienne commande
<code>ps aux grep nom</code>	trouver un processus
<code>du -sh * sort -h tail</code>	les plus gros éléments
<code>sort f uniq -c sort -rn head</code>	les 10 lignes les plus fréquentes
<code>grep -v '^#' f grep -v '^\$'</code>	configuration sans commentaires ni lignes vides
<code>awk '{print \$1}' access.log sort uniq -c sort -rn head</code>	adresses les plus fréquentes d'un journal web
<code>find . -name '*.md' -print0 xargs -0 grep -l TODO</code>	fichiers .md contenant TODO (espaces dans les noms gérés)
<code>ls wc -l</code>	nombre d'éléments du dossier
<code>ls -lt head</code>	les fichiers modifiés en dernier
<code>tail -f journal grep -i error</code>	surveiller les erreurs en direct
<code>cut -d: -f1 /etc/passwd sort</code>	liste triée des comptes

EXPRESSIONS RÉGULIÈRES (GREP -E, SED -E)

<code>.</code>	un caractère quelconque
<code>^ · \$</code>	début · fin de ligne
<code>*</code>	l'élément précédent, 0 fois ou plus
<code>+ · ?</code>	1 fois ou plus · 0 ou 1 fois
<code>{2,4}</code>	de 2 à 4 fois
<code>[abc] · [^abc]</code>	un caractère parmi · hors de
<code>[0-9] · [a-z]</code>	un caractère d'un intervalle
<code>a b · ()</code>	alternative · groupe
<code>\. · *</code>	caractère spécial pris à la lettre
<code>\b</code>	limite de mot (GNU)

EXEMPLES

<code>^#</code>	lignes de commentaire
<code>^\$</code>	lignes vides
<code>^[0-9]+\$</code>	lignes faites uniquement de chiffres
<code>[0-9]{1,3}(\.[0-9]{1,3}){3}</code>	forme d'une adresse IPv4 (valeurs non contrôlées)

❗ Joker ≠ expression régulière : `*.txt` (shell) ↔ `\.txt$` (grep).

FILTRES DE TEXTE

<code>sort f</code>	trier : <code>-n</code> numérique · <code>-r</code> inverse · <code>-h</code> tailles (2K, 1G) · <code>-k 2</code> selon la 2 ^e colonne · <code>-u</code> sans doublon
<code>uniq</code>	fusionner les doublons consécutifs (<code>-c</code> compter) : trier avant
<code>cut -d: -f1 /etc/passwd</code>	1 ^{er} champ, séparateur « : »
<code>tr a-z A-Z</code>	remplacer des caractères (ici : en majuscules)
<code>tr -d '\r'</code>	supprimer des caractères (fins de ligne Windows)
<code>sed 's/ancien/nouveau/g' f</code>	remplacer partout (affiche le résultat)
<code>sed -i.bak 's/a/b/g' f</code>	❗ modifier le fichier lui-même (copie <code>.bak</code>)
<code>sed -n '5,10p' f</code>	afficher les lignes 5 à 10
<code>sed '/^#/d' f</code>	retirer les lignes de commentaire
<code>awk '{print \$1}' f</code>	1 ^{re} colonne (séparateur : espaces)
<code>awk -F: '{print \$1, \$7}' /etc/passwd</code>	colonnes 1 et 7, séparateur « : »
<code>awk '\$3 > 100' f</code>	lignes dont la 3 ^e colonne dépasse 100
<code>tee f</code>	afficher et écrire dans un fichier
<code>xargs</code>	transformer l'entrée en arguments
<code>column -t</code>	aligner en colonnes
<code>tac · rev</code>	inverser l'ordre des lignes · des caractères

LOCALISER UNE COMMANDE OU UN FICHIER

<code>locate nom</code>	recherche instantanée dans un index (paquet <code>plocate</code>) ; mise à jour : sudo updatedb
<code>which commande</code>	programme effectivement lancé
<code>whereis commande</code>	programme, sources et manuel
<code>type -a commande</code>	toutes les définitions : alias, fonction, fichier
<code>command -v commande</code>	la commande existe-t-elle ? (forme portable, scripts)
<code>realpath f</code>	chemin absolu d'un fichier

4 · Droits et utilisateurs

LIRE UNE LIGNE DE LS -L

-

rwx

r-x

1

alice

compta

4096

Sep 28 10:15

rapport.sh

TYPE

PROPRIÉTAIRE **U**

GROUPE **G**

AUTRES **O**

LIENS

PROPRIÉTAIRE

GROUPE

TAILLE (OCTETS)

DERNIÈRE MODIFICATION

NOM

Type : - fichier · d dossier · l lien symbolique · b c périphérique (bloc, caractère) · p tube nommé · s socket. Droits dans l'ordre r lire · w écrire · x exécuter ; - = droit absent.

QUE SIGNIFIENT R, W, X ?

	SUR UN FICHIER	SUR UN DOSSIER
r (4)	lire le contenu	lister les noms (ls)
w (2)	modifier le contenu	créer, renommer, supprimer des éléments
x (1)	exécuter (programme, script)	entrer (cd) et accéder aux éléments

Supprimer un fichier dépend des droits sur le **dossier** qui le contient, pas sur le fichier lui-même.

NOTATION OCTALE

0 ---	1 --x	2 -w-	3 -wx	4 r--	5 r-x	6 rw-	7 rwx
644	rw-r--r--						fichier ordinaire
600	rw-----						fichier privé (clé SSH)
755	rxwxr-xr-x						programme, script, dossier
700	rxw-----						dossier privé (~/.ssh)
664 · 775	rw-rw-r-- · rxwxwxr-x						travail en groupe
777	rxwxrxwx						❗ tout à tous : à proscrire

Un chiffre par catégorie u · g · o ; chaque chiffre = somme de r 4 + w 2 + x 1. Ex. 6 = 4 + 2 = rw-.

MODIFIER DROITS ET PROPRIÉTAIRES

chmod 644 f	fixer les droits en octal
chmod u+x f	ajouter x au propriétaire
chmod g-w f	retirer w au groupe
chmod o= f	retirer tout droit aux autres
chmod a+r f	lecture pour tous (a = u + g + o)
chmod u=rw,go=r f	fixer chaque catégorie (= 644)
chmod -R u+rwX d	récursif ; X : x pour les dossiers seulement (et fichiers déjà exécutables)
sudo chown util f	changer le propriétaire
sudo chown util:groupe f	propriétaire et groupe
sudo chown -R util: d	récursif ; groupe principal de util
chgrp groupe f	changer le groupe (si l'on en est membre)

❗ chmod -R 777 ne résout rien : il ouvre tout à tous. Chercher plutôt qui doit accéder, et à quoi.

MASQUE ET DROITS SPÉCIAUX

umask	masque appliqué aux créations ; souvent 0022 → fichiers 644, dossiers 755 (ou 0002 → 664, 775)
chmod u+s f	setuid (4000) : le programme s'exécute avec les droits de son propriétaire (ex. passwd)
chmod g+s d	setgid (2000) sur un dossier : les nouveaux fichiers héritent de son groupe
chmod +t d	sticky (1000) : seul le propriétaire d'un fichier peut le supprimer (ex. /tmp : drwxrwxrwt)

s ou t minuscule : bit spécial avec x ; S ou T majuscule : bit spécial sans x.

UTILISATEURS ET GROUPES

whoami · id	utilisateur courant · identifiants et groupes
groups util	groupes d'un utilisateur
who · w	utilisateurs connectés (et leur activité)
passwd	changer son mot de passe
sudo adduser util	créer un utilisateur (Debian, Ubuntu : interactif)
sudo useradd -m -s /bin/bash util	créer un utilisateur (toutes distributions)
sudo passwd util	définir son mot de passe
sudo usermod -aG groupe util	ajouter à un groupe
sudo groupadd groupe	créer un groupe
sudo passwd -l util	verrouiller le mot de passe du compte
sudo userdel -r util	supprimer le compte et son dossier personnel

❗ usermod -G sans -a remplace tous les groupes secondaires. Un nouveau groupe prend effet à la connexion suivante.

DEVENIR ADMINISTRATEUR

sudo commande	exécuter une commande en administrateur (root)
sudo !!	relancer la commande précédente avec sudo
sudo -i	ouvrir un shell root (quitter : exit)
sudo -u util commande	exécuter en tant qu'un autre utilisateur
sudo -l	voir ce que l'on a le droit de faire
su - util	ouvrir une session en tant que util (son mot de passe)
sudo visudo	modifier /etc/sudoers avec contrôle de syntaxe

Invite : \$ utilisateur ordinaire · # root. Droits d'administration : groupe sudo (Debian, Ubuntu) ou wheel (Fedora, RHEL).

❗ Root n'a aucun garde-fou : réserver sudo à la seule commande qui l'exige.

FICHIERS DES COMPTES

alice	:x	:1000	:1000	:Alice	:/home/alice	:/bin/bash
NOM	MOT DE PASSE	UID	GID	INFO	DOSSIER	SHELL
/etc/passwd				comptes, une ligne chacun (lisible par tous)		
/etc/shadow				empreintes des mots de passe (root seul) ; x y renvoie		
/etc/group				groupes : nom:x:GID:membre1,membre2		
/etc/sudoers				règles de sudo (modifier via visudo)		

UID 0 = root. Comptes ordinaires : en général à partir de 1000 ; en dessous, comptes système (services).

DROITS FINS : ACL

getfacl f	afficher les droits détaillés
setfacl -m u:util:rw f	lecture et écriture pour un utilisateur précis
setfacl -m g:groupe:rx d	idem pour un groupe
setfacl -x u:util f	retirer cette entrée

Un + après les droits (-rw-r--r--+) signale des ACL.

5 · Le shell Bash

FLUX ET REDIRECTIONS

clavier
ou fichier

0 entrée standard
stdin →

commande

1 sortie standard
stdout →

2 erreur standard
stderr →

écran

écran

cmd > f

sortie vers f — ❶ écrase son contenu

cmd >> f

sortie ajoutée à la fin de f

cmd 2> f

erreurs vers f

cmd > f 2>&1

sortie **et** erreurs vers f (ordre important)

cmd &> f

idem, forme courte de Bash

cmd 2>/dev/null

jeter les erreurs

cmd < f

lire l'entrée depuis f

cmd1 | cmd2

tube : la sortie de cmd1 alimente cmd2

cmd | tee f

afficher et enregistrer à la fois

cmd <<< "texte"

une chaîne comme entrée

DOCUMENT EN LIGNE (HERE-DOC)

cat > notes.txt <<'EOF'

Texte sur plusieurs lignes,

écrit tel quel dans notes.txt.

EOF

<<'EOF' : texte pris tel quel · <<EOF : variables remplacées.

❶ **sort** f > f vide f : le shell l'ouvre en écriture **avant** la lecture. Écrire ailleurs, ou **sort -o f f**.

ENCHAÎNER LES COMMANDES

a ; b

a puis b, quoi qu'il arrive

a && b

b seulement si a **réussit**

a || b

b seulement si a **échoue**

a &

a en arrière-plan

(a ; b)

sous-shell : un **cd** n'y affecte pas le shell courant

\$(cmd)

remplacé par la sortie de cmd

mkdir -p sauv && cp -a docs "sauv/docs-\$(date +%F)"

→ copie datée **sauv/docs-AAAA-MM-JJ**, seulement si **mkdir** a réussi.

TÂCHES ET SESSIONS

cmd &

lancer en arrière-plan

jobs

lister les tâches du shell

fg %1 · bg %1

tâche 1 au premier plan · poursuivie en arrière-plan

nohup cmd &

survivre à la fermeture du terminal (sortie : **nohup.out**)

disown

détacher la dernière tâche du shell

tmux

session qui survit à la déconnexion (page 8)

HISTORIQUE

history

liste numérotée

!!

relancer la dernière commande

!n

relancer la commande numéro n

!ssh

relancer la dernière commande commençant par ssh

!\$

dernier argument de la commande précédente

^a^b

relancer la précédente en remplaçant a par b

Ctrl+R

recherche au fil de la frappe ; encore **Ctrl+R** : plus ancien

Debian, Ubuntu : une commande précédée d'une espace n'est pas mémorisée (**HISTCONTROL=ignoreboth**).

VARIABLES

nom=valeur

définir — ❶ aucune espace autour de =

echo "\$nom"

utiliser

"\${nom}_v2"

accolades : délimiter le nom

export nom=valeur

transmettre aux programmes lancés ensuite

unset nom

supprimer

env · printenv

lister les variables d'environnement

\${nom:-défaut}

valeur par défaut si vide ou absente

\${#nom}

longueur

\${f%.txt}

retirer le suffixe .txt

\${f##*/}

retirer le chemin (≈ **basename**)

\${nom/a/b}

remplacer la 1^{re} occurrence (/ : toutes)

\$((3 * 4))

calcul entier → 12

VARIABLES PRÉDÉFINIES UTILES

\$HOME · \$USER · \$PWD

dossier personnel · utilisateur · dossier courant

\$PATH

dossiers où chercher les commandes, séparés par :

\$SHELL · \$EDITOR · \$LANG

shell · éditeur par défaut · langue

\$? · \$\$

code de retour précédent · numéro (PID) du shell

GUILLEMETS ET ÉCHAPPEMENT

'...'

littéral : rien n'est interprété

echo '\$HOME' → \$HOME

"..."

variables et \$() remplacés, espaces préservées

echo "\$HOME" → /home/alice

\

neutralise le caractère suivant

echo \\$HOME → \$HOME

Règle : toujours **"\$variable"** entre guillemets doubles — sinon un nom contenant une espace se coupe en deux arguments.

CONFIGURER SON SHELL

~/ .bashrc

lu par chaque shell interactif : alias, invite, fonctions

~/ .profile · ~/ .bash_profile

lu à la connexion : variables d'environnement

/etc/profile

équivalent pour tous les utilisateurs

source ~/ .bashrc

appliquer les changements sans rouvrir le terminal

alias ll='ls -lh'

créer un raccourci (à placer dans **~/ .bashrc**)

alias · unalias nom

lister · supprimer les alias

export

ajouter un dossier au PATH

PATH="\$HOME/bin:\$PATH"

PETITS OUTILS DU QUOTIDIEN

date · date +%F

date et heure · format AAAA-MM-JJ

sleep 5

attendre 5 secondes

watch -n 2 cmd

relancer une commande toutes les 2 s

time cmd

mesurer la durée d'exécution

seq 1 10

suite de nombres de 1 à 10

echo 'scale=2; 7/2' | bc

calcul décimal → 3.50 (paquet **bc**)

reset

rétablir un terminal déréglé

xdg-open f

ouvrir avec l'application par défaut (bureau)

6 · Scripts Bash

SQUELETTE D'UN SCRIPT

```
#!/usr/bin/env bash
# sauvegarde.sh – archive datée d'un dossier
set -euo pipefail

dossier="${1:?usage : $0 dossier}"
dest="$HOME/sauvegardes"
jour="$(date +%F)" # AAAA-MM-JJ
archive="$dest/${basename "$dossier"}-$jour.tar.gz"

mkdir -p "$dest"
tar -czf "$archive" "$dossier"
echo "Archive créée dans $dest"
```

#!/usr/bin/env bash	« shebang » : l'interpréteur à utiliser (1 ^{re} ligne)
set -e	arrêter à la première commande en échec
set -u	erreur si une variable n'est pas définie
set -o pipefail	un tube échoue si l'une de ses commandes échoue
\${1:?message}	arrêt avec message si l'argument manque

LANCER

chmod +x sauvegarde.sh	rendre exécutable (une fois)
./sauvegarde.sh docs	lancer depuis son dossier
bash sauvegarde.sh docs	lancer sans le droit x

❶ ./ est nécessaire : le dossier courant n'est pas dans \$PATH.

❷ Script écrit sous Windows (fins de ligne CRLF) → **bad interpreter** ou \$'\r': **command not found**. Corriger : **sed -i 's/\r\$//'** script ou dos2unix script.

ARGUMENTS ET VARIABLES SPÉCIALES

\$0	nom du script
\$1 ... \$9 · \${10}	arguments de position
##	nombre d'arguments
"\$@"	tous les arguments, chacun intact (à préférer)
"\$*"	tous les arguments en une seule chaîne
\$?	code de retour de la dernière commande
\$\$ · \$!	PID du script · du dernier processus lancé en arrière-plan
shift	décaler : \$2 devient \$1
exit 0 · exit 1	terminer : succès · échec

TESTS : [[...]]

FICHIERS

-e f	existe
-f f · -d d	fichier ordinaire · dossier
-L f	lien symbolique
-r · -w · -x f	lisible · modifiable · exécutable
-s f	existe et n'est pas vide
f1 -nt f2	f1 plus récent que f2

CHAÎNES

-z "\$s" · -n "\$s"	vide · non vide
"\$a" == "\$b" · !=	égales · différentes
"\$f" == *.txt	correspond au joker (sans guillemets à droite)
"\$s" =~ ^[0-9]+\$	correspond à l'expression régulière

NOMBRES

-eq · -ne	égal · différent
-lt · -le · -gt · -ge	< · ≤ · > · ≥
((a > b))	comparaison arithmétique, plus lisible

Combiner : &&, ||, ! à l'intérieur de [[]]. ❶ Espaces obligatoires : [[-f "\$f"]], jamais [[-f "\$f"]].
[] (POSIX) : mêmes tests de fichiers, = pour les chaînes, guillemets indispensables.

CONDITIONS

```
if [[ -f "$f" ]]; then
    echo "fichier présent"
elif [[ -d "$f" ]]; then
    echo "c'est un dossier"
else
    echo "introuvable : $f" >&2
    exit 1
fi

case "$reponse" in
    o|O|oui) echo "d'accord" ;;
    n|N|non) echo "abandon"; exit 1 ;;
    *)      echo "réponse inattendue" ;;
esac
```

Toute commande peut servir de condition : code 0 = vrai.
if grep -q motif f; then ...

BOUCLES

```
for f in *.jpg; do
    echo "Traitement de $f"
done

for i in {1..5}; do echo "$i"; done
for (( i = 0; i < 3; i++ )); do echo "$i"; done

while IFS= read -r ligne; do
    echo "> $ligne"
done < fichier.txt

n=0
until (( n >= 3 )); do n=$(( n + 1 )); done

break sortir de la boucle · continue passer au tour suivant. Lire un fichier ligne à ligne : while IFS= read -r (espaces et \ conservés).
```

FONCTIONS ET SAISIE

```
saluer() {
    local nom="${1:-inconnu}" # variable locale
    echo "Bonjour, $nom"
}

saluer "le monde"
message="$(saluer "vous")" # capturer la sortie

read -rp "Nom : " nom      demander une saisie
printf '%s : %d\n' "$nom"  affichage formaté "$n"
return n                   code de retour d'une fonction (0 à 255)
```

DÉBOGUEUR

bash -n script	vérifier la syntaxe sans exécuter
bash -x script	afficher chaque commande exécutée
set -x · set +x	activer · couper la trace dans le script
shellcheck script	détecter erreurs et pièges (paquet shellcheck)
echo "debug : \$var" >&2	message vers la sortie d'erreur
tmp="\$(mktemp)"	fichier temporaire sûr
trap 'rm -f "\$tmp"' EXIT	nettoyer à la sortie, même en cas d'erreur

BONNES PRATIQUES

- Guillemets doubles autour de chaque variable : "\$f".
- [[]] plutôt que [] en Bash ; \$() plutôt que les accents graves.
- Variables en minuscules : les majuscules désignent l'environnement (PATH, HOME).
- Tester sur des copies ; préfixer d'un **echo** une commande destructrice pour voir ce qu'elle ferait.
- Script lancé par cron : chemins absolus (page 7).
- ❶ Avec **set -e**, ((n++)) arrête le script quand n vaut 0 (l'expression vaut « faux ») → n=\$((n + 1)).

7 · Système : processus, services, paquets

PROCESSUS

ps aux	tous les processus : utilisateur, PID, %CPU, %MEM, commande
ps -ef --forest · pstree	avec la filiation parent → enfant
pgrep -a nom	PID et ligne de commande, par nom
top	moniteur : P tri CPU · M tri mémoire · q quitter
htop	moniteur plus lisible (à installer)
kill PID	demande l'arrêt (signal TERM)
kill -9 PID	forcer l'arrêt (KILL) : dernier recours
pkill nom · killall nom	arrêter par nom
nice -n 10 cmd	lancer avec une priorité plus basse
renice -n 10 -p PID	changer la priorité (-20 prioritaire ... 19 discret)
lsuf f · lsuf -i :port	qui utilise ce fichier · ce port

SIGNAUX

N°	NOM	EFFET
1	HUP	raccrochage ; souvent : relire la configuration
2	INT	interruption (Ctrl+C)
9	KILL	arrêt immédiat, impossible à intercepter
15	TERM	demande d'arrêt propre (défaut de kill)
18	CONT	repandre un processus suspendu
19	STOP	suspendre (impossible à intercepter)
20	TSTP	suspension au clavier (Ctrl+Z)

Liste complète : **kill -l**. Numéros valables sur PC (x86) et ARM.

PLANIFIER : CRON

```
----- minute (0-59)
|----- heure (0-23)
| |----- jour du mois (1-31)
| | |----- mois (1-12)
| | | |----- jour de la semaine (0-7 ; 0 et 7 = dimanche)
| | | | |
* * * * * commande
```

30 2 * * * /chemin/script.sh	chaque jour à 2 h 30
* /15 * * * * cmd	toutes les 15 minutes
0 9 * * 1-5 cmd	à 9 h, du lundi au vendredi
0 0 1 * * cmd	le 1 ^{er} de chaque mois, à minuit
@reboot cmd	à chaque démarrage
crontab -e · crontab -l	modifier · afficher sa table
crontab -r	❗ supprime toute la table, sans confirmation (voisin de -e)

❗ Environnement minimal : chemins absolus, sortie redirigée
(>> /chemin/Log 2>&1), % écrit %.

INFORMATIONS SYSTÈME

uname -a · uname -r	noyau et architecture · version du noyau
cat /etc/os-release	distribution et version
hostnamectl	nom de la machine, système, matériel
uptime	durée de fonctionnement ; charge moyenne sur 1, 5, 15 min
free -h	mémoire vive et swap
lscpu · nproc	processeur · nombre de cœurs
lspci · lsusb	périphériques PCI · USB
sudo dmesg -T	messages du noyau (matériel, pilotes)
timedatectl	date, heure, fuseau, synchronisation

SERVICES : SYSTEMD

systemctl status service	état et dernières lignes du journal
sudo systemctl start service	démarrer (stop : arrêter)
sudo systemctl restart service	redémarrer
sudo systemctl reload service	relire la configuration sans couper
sudo systemctl enable --now service	activer au démarrage et lancer aussitôt
sudo systemctl disable service	ne plus lancer au démarrage
systemctl is-active service	réponse courte (is-enabled : activé ?)
systemctl --failed	services en échec
systemctl list-units --type=service	services chargés
systemctl list-timers	minuteries (alternative à cron)
sudo systemctl daemon-reload	après modification d'un fichier d'unité
sudo systemctl reboot	redémarrer (poweroff : éteindre)

Fichiers d'unité : /etc/systemd/system/ (administrateur) l'emporte sur /usr/lib/systemd/system/ (paquets).

JOURNAUX

journalctl -u service	journal d'un service
journalctl -f	suivre en direct
journalctl -b · -b -1	depuis le démarrage · démarrage précédent
journalctl -p err	erreurs et plus grave
journalctl --since today	depuis minuit (ou "1 hour ago")
journalctl -xe	fin du journal, avec explications
journalctl --disk-usage	place occupée par le journal

Fichiers texte (si rsyslog est installé) : /var/log/syslog (Debian) ou /var/log/messages (Fedora, RHEL) ; connexions et sudo : /var/log/auth.log ou /var/log/secure.

PAQUETS

ACTION	DEBIAN, UBUNTU	FEDORA, RHEL	ARCH
Mettre à jour la liste	sudo apt update	automatique (dnf makecache)	compris dans -Syu
Tout mettre à jour	sudo apt upgrade	sudo dnf upgrade	sudo pacman -Syu
Installer	sudo apt install p	sudo dnf install p	sudo pacman -S p
Installer un fichier local	sudo apt install ./f.deb	sudo dnf install ./f.rpm	sudo pacman -U f.pkg.tar.zst
Supprimer	sudo apt remove p (purge : + config.)	sudo dnf remove p	sudo pacman -Rns p
Dépendances devenues inutiles	sudo apt autoremove	sudo dnf autoremove	pacman -Qdtq sudo pacman -Rns -
Rechercher · informations	apt search mot · apt show p	dnf search mot · dnf info p	pacman -Ss mot · pacman -Si p
Paquets installés	apt list --installed	dnf list --installed	pacman -Q
Quel paquet fournit ce fichier ?	dpkg -S /chemin	dnf provides /chemin	pacman -Qo /chemin

Formats universels : **flatpak install ... · snap install ...** ❗ N'ajouter de dépôt tiers qu'en connaissance de cause : il reçoit les droits de root à chaque mise à jour.

8 · Réseau, SSH et dépannage

RÉSEAU : ÉTAT ET DIAGNOSTIC

<code>ip a · ip -br a</code>	interfaces et adresses IP · résumé
<code>ip r</code>	routes ; passerelle : ligne <code>default via ...</code>
<code>ping -c 4 hôte</code>	joignabilité (4 essais)
<code>tracpath hôte</code>	chemin suivi jusqu'à l'hôte
<code>ss -tulpn</code>	ports en écoute (<code>sudo</code> : avec tous les processus)
<code>dig nom · dig +short nom</code>	interroger le DNS
<code>resolvectl status</code>	serveurs DNS utilisés (systemd-resolved)
<code>curl -I https://example.com</code>	en-têtes de la réponse HTTP
<code>curl -LO url · wget url</code>	télécharger un fichier
<code>nmcli device status</code>	interfaces gérées par NetworkManager
<code>/etc/hosts · /etc/resolv.conf</code>	noms locaux · DNS configurés

DÉMARCHE, DE PROCHE EN PROCHE

- `ip a` : ai-je une adresse ?
- `ip r` : ai-je une passerelle ? `ping` vers elle
- `ping -c 3 1.1.1.1` : Internet répond-il ?
- `dig nom` : le DNS répond-il ?
- `curl -I url` : le service répond-il ?

SSH : ADMINISTRER À DISTANCE

<code>ssh util@hôte</code>	se connecter
<code>ssh -p 2222 util@hôte</code>	port non standard
<code>ssh util@hôte 'cmd'</code>	exécuter une commande à distance
<code>exit · Ctrl+D</code>	se déconnecter
<code>ssh-keygen -t ed25519</code>	créer une paire de clés (<code>~/.ssh/id_ed25519</code> et <code>.pub</code>)
<code>ssh-copy-id util@hôte</code>	installer sa clé publique sur le serveur
<code>ssh -L 8080:localhost:80 util@hôte</code>	tunnel : port local 8080 → port 80 du serveur
<code>ssh -J rebond util@hôte</code>	passer par une machine de rebond

RACCOURCI : `~/.SSH/CONFIG`

```
Host srv
  HostName 203.0.113.10
  User util
  Port 2222
  IdentityFile ~/.ssh/id_ed25519
```

→ `ssh srv` suffit.

❗

La clé **privée** (sans `.pub`) ne se transmet jamais. Droits : `chmod 700 ~/.ssh · chmod 600 ~/.ssh/id_ed25519`.

❗

« REMOTE HOST IDENTIFICATION HAS CHANGED » : ne pas passer outre ; vérifier l'empreinte du serveur avant `ssh-keygen -R hôte`.

TRANSFÉRER DES FICHIERS

<code>scp f util@hôte:/chemin/</code>	copier vers le serveur
<code>scp util@hôte:/chemin/f .</code>	copier depuis le serveur
<code>rsync -avh src/ util@hôte:dest/</code>	synchroniser : seules les différences transitent
<code>rsync -avh --dry-run ...</code>	simuler d'abord
<code>rsync -avh --delete src/ dest/</code>	❗ miroir : efface dans <code>dest</code> ce qui a disparu de <code>src</code>
<code>sftp util@hôte</code>	session interactive : <code>put, get, ls, bye</code>

`rsync` : `-a` archive (récuratif, droits, dates, liens) · `-v` détails · `-h` tailles lisibles · `-z` compresser · `-P` progression et reprise.

❗ `src/` copie le **contenu** de `src` ; `src` sans barre finale copie le dossier lui-même.

PORTS COURANTS

21	FTP	22	SSH, SFTP	25	SMTP
53	DNS	80	HTTP	123	NTP
143	IMAP	443	HTTPS	587	SMTP (envoi)
993	IMAPS	995	POP3S	3306	MySQL
3389	RDP	5432	PostgreSQL	8080	HTTP alt.

Liste de référence locale : `/etc/services`.

DURCIR SSH ET PARE-FEU

Dans `/etc/ssh/sshd_config`, après avoir vérifié que la connexion par clé fonctionne :

```
PasswordAuthentication no
PermitRootLogin no
```

<code>sudo sshd -t</code>	vérifier la syntaxe de la configuration
<code>sudo systemctl reload ssh</code>	appliquer (service <code>sshd</code> sur Fedora, RHEL)
<code>sudo ufw allow 22/tcp</code>	pare-feu <code>ufw</code> (Debian, Ubuntu) : autoriser SSH...
<code>sudo ufw enable · ufw status</code>	... puis activer · contrôler
<code>sudo firewall-cmd --list-all</code>	firewalld (Fedora, RHEL) : état
<code>sudo firewall-cmd --permanent --add-service=http --reload</code>	ouvrir un service, puis --reload

❗ Garder une session ouverte et tester depuis une seconde fenêtre avant de la fermer. Un fichier de `/etc/ssh/sshd_config.d/` peut primer : la première valeur lue l'emporte.

MESSAGES D'ERREUR FRÉQUENTS

command not found	faute de frappe, paquet absent, dossier hors du PATH → type , installer
No such file or directory	chemin ou casse erronés → <code>pwd, ls, Tab</code> ; script : fins de ligne CRLF
Permission denied	droits insuffisants, x manquant → <code>ls -l, chmod +x, sudo</code> si légitime
Is a directory	option <code>-r</code> oubliée ou mauvais argument
No space left on device	disque ou inodes pleins → <code>df -h, df -i, du -sh *</code>
Read-only file system	montage en lecture seule, souvent après une erreur disque → <code>sudo dmesg -T</code>
Device or resource busy	fichier ou montage en cours d'usage → <code>ls -o</code>
Connection refused	aucun service n'écoute ce port → <code>ss -tulpn, systemctl status</code>
Connection timed out	hôte injoignable ou pare-feu → <code>ping, ip r</code>
Address already in use	port déjà occupé → <code>sudo ss -tulpn</code>
Could not get lock ...	une autre installation tourne → attendre ; ne pas effacer le verrou
... is not in the sudoers file	compte sans droits d'administration → groupe <code>sudo</code> ou <code>wheel</code>

TMUX : SESSIONS PERSISTANTES

<code>tmux new -s nom</code>	ouvrir une session nommée
<code>Ctrl+B puis D</code>	se détacher : tout continue de tourner
<code>tmux ls</code>	lister les sessions
<code>tmux attach -t nom</code>	retrouver une session
<code>Ctrl+B puis C · N</code>	nouvelle fenêtre · fenêtre suivante
<code>Ctrl+B puis % · "</code>	partager l'écran : côte à côte · l'un sous l'autre
<code>Ctrl+B puis ↑ ↓ ← →</code>	changer de volet

Sur un serveur : un travail long lancé dans `tmux` survit à une coupure de la connexion SSH.